Das 30-Tage Framework

Wie du Nebenprojekte endlich fertig baust und in 30 Tagen eine App veröffentlichst (auch ohne viel Programmierwissen)

Von
Kevin Chromik
AppBuilderCircle.dev

VORWORT

Kennst du das? Du hast eine Idee für eine App, eine richtig gute sogar, und du siehst sie schon vor dir, stellst dir vor, wie Leute sie nutzen und vielleicht sogar dafür bezahlen. Die Motivation ist da, du schaust dir ein paar Tutorials an, liest dich ein und vielleicht kaufst du sogar einen Kurs. Du sagst dir: Diesmal ziehe ich es wirklich durch.

Dann fängst du an und die ersten Tage laufen gut. Aber irgendwann merkst du, wie viel da eigentlich noch kommt, das Leben fordert seinen Platz zurück und der Alltag holt dich ein. Ehe du dich versiehst, liegt das Projekt wieder auf Eis, die nächste Idee klopft schon an und der Kreislauf beginnt von vorn.

Falls dir das bekannt vorkommt, bist du damit nicht allein. Mir ging es jahrelang genauso. Das eigentliche Problem war nicht mangelnde Disziplin oder fehlendes Talent, sondern die Art, wie ich an Nebenprojekte herangegangen bin. Ich habe sie behandelt wie Vollzeit-Projekte mit allem Drum und Dran, und genau das funktioniert nicht, wenn du abends nach der Arbeit oder am Wochenende ein paar Stunden Zeit hast.

Dieses E-Book stellt dir ein Framework vor, das ich über die Jahre entwickelt und verfeinert habe. Es hilft mir dabei, Ideen schnell auf ihre Machbarkeit zu prüfen, ihren tatsächlichen Nutzen zu testen und sie dann wirklich zu veröffentlichen. Nicht irgendwann, sondern innerhalb von 30 Tagen.

Kevin Chromik ist Softwareentwickler mit über zehn Jahren Berufserfahrung, Indie-App-Entwickler und Content Creator. Er baut iOS und macOS Apps, veröffentlicht eigene Produkte und zeigt offen, was in der Praxis funktioniert und was nicht.

SCHNELLSTART: WAS DU IN 30 TAGEN ERREICHT HABEN WIRST

Wenn du dieses E-Book liest, willst du keine Theorie, sondern ein Ergebnis.

In 30 Tagen wirst du eine App veröffentlicht haben. Keine perfekte App und keine riesige Plattform, sondern ein echtes Produkt im App Store, das man benutzen kann, zu dem du Feedback bekommst und das du weiterentwickeln kannst.

Dieses Framework ist dafür gebaut, dass du es neben deinem Alltag durchziehen kannst. Mit wenig Zeit, mit unregelmäßiger Energie und ohne dass du jedes Mal beim gleichen Punkt wieder aufgibst.

Am Ende dieser 30 Tage hast du:

- Eine App im App Store, Version 1.0
- Ein Kernfeature, das vollständig funktioniert
- Eine klare Entscheidung, was Version 2.0 wird und was nicht
- App Store Texte, Screenshots und ein fertiges Icon
- Erste echte Rückmeldungen von Nutzern
- Ein System, das du für jede neue App-Idee wiederholen kannst

Wichtig: Das Ziel ist nicht, in 30 Tagen reich zu werden, sondern endlich zu shippen.

Wenn du bisher oft Projekte gestartet und nie beendet hast, dann liegt das nicht daran, dass du unfähig bist, sondern daran, dass dir ein Prozess gefehlt hat, der dich durch den schwierigen Mittelteil bringt.

Die 5 Schritte zum Launch:

Schritt	Ziel	Tage
1. Konzept	Idee validieren	Tag 1-4
2. MVP	Kernfunktionen	Tag 5-9
3. Build	Umsetzung	Tag 10-19
4. Beta	Erste Tester	Tag 20-25
5. Launch	Veröffentlichen	Tag 26-30

Die wichtigste Regel des ganzen E-Books:

- Du baust nur Version 1.0. Nicht deine Traum-App.

Alles, was du nach Schritt 2 neu erfindest, ist fast immer Scope Explosion, und Scope Explosion ist der Grund, warum die meisten Nebenprojekte scheitern.

Du brauchst keine Motivation, sondern Momentum.

Warum Nebenprojekte scheitern – und was du anders machen musst

Die meisten Nebenprojekte scheitern nicht an der Idee, sondern an der Umsetzung. Der typische Ablauf: Du bist begeistert, fängst an zu bauen, merkst dass alles komplizierter ist als gedacht, und irgendwann liegt das Projekt auf Eis.

Das passiert, wenn Nebenprojekte scheitern:

- Der Umfang war von Anfang an zu groß
- Du hast zu früh angefangen, Dinge zu perfektionieren
- Es gab keine feste Deadline
- Du hast Features gebaut, ohne zu wissen, ob sie jemand braucht

Der Unterschied zu Vollzeitprojekten ist entscheidend: Im Job hast du feste Arbeitszeiten, Team-Support, Deadlines und klare Prozesse. Bei einem Nebenprojekt hast du Zeit nach Feierabend, du bist allein, niemand fragt nach, und es gibt tausend Ablenkungen.

Nebenprojekte brauchen einen radikal einfachen Ansatz. Eine Struktur, die so simpel ist, dass du sie auch dann durchziehen kannst, wenn du müde bist und eigentlich keine Lust hast.

Das 30-Tage-Framework lässt sich in einem Satz zusammenfassen:

- **In 5 klaren Schritten – Konzept, MVP, Build, Beta, Launch – bringst du eine App vom ersten Gedanken in den App Store.**

Es geht um drei Dinge:

1. Momentum aufbauen – Jeden Tag ein Schritt nach vorn
2. Klarheit gewinnen – Wissen, was wichtig ist und was nicht
3. Tatsächlich launchen – Eine echte App im Store

Die 3 Regeln für einen Launch in 30 Tagen

Diese drei Regeln sind nicht optional, sondern der Unterschied zwischen einem Projekt, das fertig wird, und einem, das auf der Festplatte verrottet.

Regel 1: Eine Kernfunktion
Dein MVP hat genau eine Kernfunktion. Nicht zwei, nicht drei. Eine.
Frag dich: Was ist die eine Sache, die meine App können muss, damit sie einen Wert hat? Alles andere ist Nice-to-have und kommt frühestens nach dem Launch.

Regel 2: Scope Freeze nach Schritt 2
Nach Tag 9 sind keine neuen Features mehr erlaubt.
Alles, was bis Ende Schritt 2 (MVP) feststeht, wird gebaut. Jede neue Idee danach kommt auf eine Liste für Version 2.0.

Regel 3: Release ist Pflicht
Am Tag 30 wird veröffentlicht. Keine Ausnahmen.
Nicht “wenn es fertig ist” und nicht “wenn ich noch dieses eine Feature habe”. Das zwingt dich dazu, Prioritäten zu setzen und nur das zu bauen, was wirklich nötig ist.

Regel	**Was sie bedeutet**
Eine Kernfunktion	Fokus statt Feature-Hölle
Scope Freeze nach Schritt 2	Keine neuen Ideen mehr
Release ist Pflicht	Tag 30 wird geliefert

Schritt 1: Konzept – Idee validieren (Tag 1-4)

Die ersten vier Tage haben ein klares Ziel: Herausfinden, ob deine Idee realistisch umsetzbar ist. Nicht irgendwann, sondern jetzt, als Solo-Entwickler, neben deinem normalen Leben.

Was du am Ende von Schritt 1 hast: Eine klare Idee in einem Satz, ein definiertes Kernfeature, bestätigte technische Machbarkeit und eine Go/No-Go Entscheidung.

Ziel in einem Satz (Tag 1)
Du formulierst das Ziel deiner App in genau einem Satz. Dieser Satz zwingt dich dazu, das Wesentliche zu destillieren.

Beispiele: - “Der Nutzer kann täglich eine Aufgabe eintragen und abhaken.” - “Der Nutzer scannt eine Quittung und bekommt eine Liste seiner Ausgaben.”

- **Wenn du mehr als einen Satz brauchst, um zu erklären, was deine App tut, ist sie wahrscheinlich zu komplex für ein Nebenprojekt.**

Kernfeature definieren (Tag 1-2)
Frag dich: Was ist die eine Sache, die meine App können muss, damit sie einen Wert hat?
Schreib es auf. Das ist dein Kernfeature. Alles andere kommt auf eine “Später”-Liste.

Risiken erkennen und testen (Tag 2-4)
Geh durch deine Idee und frag dich bei jedem Teil: Weiß ich, wie das funktioniert? Habe ich das schon mal gemacht?

Frage	Risiko-Level
Brauche ich ein Login-System?	Hoch
Brauche ich Sync zwischen Geräten?	Hoch
Brauche ich eine externe API?	Mittel
Brauche ich ein Bezahlsystem?	Hoch

Nimm das größte Risiko und baue den kleinstmöglichen Test dafür. Kein schönes UI, nur die nackte Funktion. Wenn der Test funktioniert, weißt du, dass die App machbar ist. Wenn nicht, hast du nur zwei Tage investiert statt zwei Wochen.

Go oder No-Go (Tag 4)
Go: Idee ist klar, Kernfeature ist definiert, technisch machbar.
No-Go: Zu viele offene technische Fragen, im Rahmen eines Nebenprojekts nicht lösbar.

Beide Entscheidungen sind gut – die einzig schlechte Entscheidung wäre, keine zu treffen.

eckliste Schritt 1

- [ ] Idee ausgewählt
- [ ] Ziel in einem Satz formuliert
- [ ] Kernfeature festgelegt
- [ ] Risiken identifiziert
- [ ] Größtes Risiko getestet
- [ ] Go/No-Go Entscheidung getroffen

Schritt 2: MVP – Kernfunktionen (Tag 5-9)

In Schritt 2 definierst du den genauen Umfang deiner App und baust einen ersten funktionierenden Prototyp. Am Ende dieser Phase steht der **Scope Freeze**.

Was du am Ende von Schritt 2 hast: Einen funktionierenden Prototyp, echtes Feedback von 3-5 Personen, eine finale Must-Have Liste und einen eingefrorenen Scope.

Prototyp bauen (Tag 5-7)
Du arbeitest im Hackathon-Modus: Geschwindigkeit vor Perfektion.

1. **Der Code muss nicht schön sein** – Er muss nur funktionieren.
2. **Vergiss perfekte Architektur** – Eine simple Struktur reicht völlig.
3. **Design bleibt außen vor** – Nutze System-Buttons, Standard-Listen.

Am Ende von Tag 7 solltest du etwas haben, das du auf dein Handy laden und benutzen kannst.

Feedback einholen (Tag 8)
Schick deinen Prototyp an 3-5 Personen und stell diese drei Fragen:

Frage	Was du lernst
Würdest du das nutzen?	Grundsätzliches Interesse
Was fehlt dir?	Wichtigste Lücken
Würdest du zahlen? Wie viel?	Monetarisierungspotenzial

Scope Freeze (Tag 9)
Der wichtigste Tag des ganzen Frameworks.

An Tag 9 entscheidest du final: - Was kommt in Version 1.0? → Must-Have Liste - Was kommt später? → Nice-to-Have Liste

- **Ab Tag 9 sind keine neuen Features mehr erlaubt.**

Checkliste Schritt 2

- [ ] Prototyp gebaut
- [ ] Kernfeature funktioniert
- [ ] 3-5 Personen um Feedback gebeten
- [ ] Must-Have Liste finalisiert
- [ ] Nice-to-Have Liste geschrieben
- [ ] **Scope eingefroren**

Schritt 3: Build – Umsetzung (Tag 10-19)

Jetzt wird gebaut. Du hast einen validierten Prototyp und einen eingefrorenen Scope. In den nächsten 10 Tagen machst du aus dem Prototyp ein stabiles Produkt.

Was du am Ende von Schritt 3 hast: Ein vollständig funktionierendes MVP, stabiles Kernfeature ohne Crashes, Onboarding für neue Nutzer. Vom Prototyp zum Produkt

Du baust nur, was auf der Must-Have Liste steht.

- Keine neuen Features
- Keine grundlegenden Änderungen an der Architektur
- Keine Design-Experimente

Tag	Fokus
Tag 10	Projekt aufräumen, Struktur verbessern
Tag 11-12	Kernfeature stabilisieren und komplett machen
Tag 13	Edge Cases abfangen
Tag 14	Einstellungen (nur wenn nötig)
Tag 15	Onboarding und leere Zustände
Tag 16	UX-Verbesserungen (Texte, Buttons)
Tag 17-18	Erster kompletter Durchlauf, Bugs fixen
Tag 19	Stabilitätstag

Was ein MVP wirklich ist

Ein MVP ist die kleinstmögliche Version deiner App, die:

- Einen echten Nutzen liefert
- Feedback von Nutzern erzeugen kann
- Als eigenständiges Produkt funktioniert

Ein MVP ist NICHT: Eine halbfertige Beta, eine Demo mit “Coming Soon”-Hinweisen, oder ein Prototyp mit Buttons, die nichts tun.

Checkliste Schritt 3

- [ ] Kernfeature vollständig und stabil
- [ ] Alle relevanten Edge Cases abgefangen
- [ ] Onboarding eingebaut
- [ ] App läuft ohne Crashes
- [ ] Erster kompletter Durchlauf erfolgreich

Schritt 4: Beta – Erste Tester (Tag 20-25)

Du hast jetzt eine funktionsfähige App. Bevor du in den App Store gehst, brauchst du echte Menschen, die die App auf ihren Geräten testen.

Was du am Ende von Schritt 4 hast: App-Icon und Name finalisiert, TestFlight Build live, Feedback von echten Testern, kritische Bugs gefixt, App Store Texte und Screenshots.

Tag	Fokus
Tag 20	Design Minimal-Polish (Abstände, Farben, Typo)
Tag 21	App-Icon erstellen, Name finalisieren
Tag 22	TestFlight Build erstellen und hochladen
Tag 23	5-10 Tester einladen, Feedback-Kanal einrichten
Tag 24	Kritische Bugs fixen, wichtigstes Feedback einarbeiten
Tag 25	App Store Texte schreiben, Screenshots erstellen

Wichtig: Nur kritische Bugs fixen. Keine neuen Features!

Checkliste Schritt 4

- [ ] Design minimal poliert
- [ ] App-Icon erstellt
- [ ] Name finalisiert
- [ ] TestFlight Build live
- [ ] 5-10 Tester haben die App
- [ ] Kritische Bugs gefixt
- [ ] App Store Texte geschrieben
- [ ] Screenshots erstellt

Schritt 5: Launch – Veröffentlichen (Tag 26-30)

Die letzten 5 Tage. Die App ist fertig, getestet und bereit für die Welt.

Für viele ist das der schwierigste Schritt – nicht technisch, sondern emotional. Die App fühlt sich noch nicht “fertig” an...

Nein. Du veröffentlichst am Tag 30. Eine unvollkommene App im Store ist unendlich mehr wert als eine perfekte App auf deiner Festplatte.

Was du am Ende von Schritt 5 hast: Eine App im App Store, erste echte Downloads, einen Plan für Version 2.0.

Tag	Fokus
Tag 26	Store Listing in App Store Connect
Tag 27	Finale Screenshots, letzter Review-Check
Tag 28	App Review einreichen
Tag 29	Launch-Post vorbereiten
Tag 30	**LIVE GEHEN** – Release, Post absetzen, Feedback sammeln

Bei Rejection: Sofort fixen, nicht diskutieren. Lies die Begründung, behebe das Problem und reiche erneut ein.

Checkliste Schritt 5

- [ ] Store Listing komplett
- [ ] Screenshots final
- [ ] App eingereicht
- [ ] Review bestanden
- [ ] Launch-Post vorbereitet
- [ ] **App ist live**
- [ ] Erstes Update geplant

No-Go-Liste: Ideen und Features, die dich killen

Nicht jede App-Idee eignet sich für ein 30-Tage-Projekt, und nicht jedes Feature gehört in Version 1.0.

App-Ideen, die in 30 Tagen fast immer scheitern

Idee	Warum es scheitert
Social Network	Braucht kritische Masse, Server-Infrastruktur, Moderation
Marktplatz	Zwei-Seiten-Problem: Käufer UND Verkäufer nötig
Messenger	Netzwerkeffekte, Datenschutz, Echtzeit-Infrastruktur
Vollständige Finanz-App	Bank-APIs, Regulierung, Sicherheitsanforderungen
Spiel mit Multiplayer	Server, Matchmaking, Cheating-Prevention

Features, die NICHT in Version 1.0 gehören

Feature	Warum es Version 1.0 killt
Login/Account-System	Komplexität explodiert
Cloud-Sync	Server-Kosten, Konfliktauflösung
Soziale Features	Jedes ein eigenes Projekt
Vollständige Einstellungen	20 Optionen = 10x mehr Arbeit
Alle Plattformen gleichzeitig	Erst eine, dann erweitern

Die Faustregel

- **Wenn ein Feature einen eigenen Unterabschnitt in der Dokumentation braucht, gehört es nicht in Version 1.0.**

Was stattdessen tun?

Statt	Mach das
Cloud-Sync	Lokale Speicherung + Export
Login-System	Daten bleiben auf dem Gerät
Soziale Features	Share-Button für Screenshots
Komplexe Einstellungen	Sinnvolle Defaults
Alle Plattformen	Nur iOS (oder nur Android)

Beispiel-App: Fokus-App für Menschen mit ADHS

Schwierigkeit: Einfach – Perfekt für Anfänger

Das Problem
Menschen mit ADHS haben oft Schwierigkeiten, sich auf eine Aufgabe zu konzentrieren. Klassische Produktivitäts-Apps sind überladen mit Features, was das Problem eher verschlimmert. Was gebraucht wird: maximale Einfachheit.

Die Lösung
Eine App, die nur eines tut: dir helfen, dich auf genau eine Aufgabe zu konzentrieren. Du gibst ein, woran du arbeiten willst, startest einen Timer und die App zeigt dir während der gesamten Zeit nur diese eine Aufgabe an. Groß, klar und ohne Ablenkung.

MVP für 30 Tage
- Eine Aufgabe eingeben und als aktiv setzen
- Timer starten (z.B. 25 Minuten Fokuszeit)
- Während der Fokuszeit: Nur die aktive Aufgabe groß auf dem Screen
- Sound/Vibration wenn die Zeit vorbei ist
- Einfache Statistik: Wie viele Fokus-Sessions heute?

Was NICHT in Version 1.0 gehört
- Keine Aufgabenlisten oder Projektverwaltung
- Keine Integration mit anderen Apps
- Keine komplexen Statistiken oder Gamification
- Kein Account-System

Worauf du achten solltest
Diese App ist perfekt für den Einstieg: keine Netzwerk-Requests, keine Synchronisation, keine komplexe Datenhaltung. Der Timer ist die einzige Komponente, die Aufmerksamkeit braucht – er muss auch im Hintergrund weiterlaufen.

Der größte Fehler wäre, die App “aufzuwerten” mit Features wie Kategorien oder Statistik-Dashboards. Der ganze Wert liegt in der Einfachheit.

Monetarisierung
Einmal-Kauf 2,99€ oder Pro-Abo 1,99€/Monat für zusätzliche Timer-Presets, Sounds und ein Home-Screen-Widget.

Nächster Schritt

Du hast jetzt alles, was du brauchst, um deine erste App in 30 Tagen zu bauen. Warte nicht auf den perfekten Moment. Fang heute an.

Alleine zu arbeiten ist möglich – aber es ist auch härter als nötig. Was oft fehlt: Struktur, Momentum und echtes Feedback von Menschen, die verstehen, was du baust.

Deshalb gibt es den **App Builder Circle**: Eine Community von Menschen, die Apps tatsächlich fertigstellen und veröffentlichen. Mit Challenges, Feedback-Runden und Q&A-Sessions.

Egal ob alleine oder mit Support: Der wichtigste Schritt ist, HEUTE anzufangen.

Abschluss

Wenn du nur eine einzige Sache aus diesem E-Book mitnimmst, dann diese:

- **Die meisten Apps scheitern nicht am Code. Sie scheitern nicht an fehlenden Skills oder schlechten Ideen. Sie scheitern daran, dass sie nie veröffentlicht werden.**

Motivation ist flüchtig und unzuverlässig. Was du brauchst, ist ein System. Das 30-Tage-Framework ist dieses System:

Schritt	Ziel	Tage
1. Konzept	Idee validieren	Tag 1-4
2. MVP	Kernfunktionen	Tag 5-9
3. Build	Umsetzung	Tag 10-19
4. Beta	Erste Tester	Tag 20-25
5. Launch	Veröffentlichen	Tag 26-30

Der schwierigste Teil ist nicht das Bauen, sondern das Anfangen.

1. Such dir eine Idee aus
2. Starte heute mit Tag 1
3. Folge den 5 Schritten

In 30 Tagen könntest du eine App im Store haben. Oder du könntest immer noch über die perfekte Idee nachdenken. Die Entscheidung liegt bei dir.

Viel Erfolg dabei.